Universidad Católica San Pablo

PROYECTO BDII

Indexación y Distribución de Datos en PostgreSQL

El proyecto se trabajará a lo largo de diez semanas, en dos etapas. Las primeras cinco semanas estarán dedicadas al trabajo de indexación y se evaluarán en el examen parcial práctico. Durante las cinco semanas siguientes se completará la evaluación del índice y se trabajará la parte distribuida, que formará parte de la evaluación final.

La idea es que el proyecto permita aplicar progresivamente los temas vistos en el curso. Se espera una implementación académica funcional, bien explicada y reproducible; no una implementación con el nivel de complejidad de un componente de PostgreSQL listo para producción.

1. Propósito del proyecto

El objetivo es llevar a la práctica los temas de diseño físico y bases de datos distribuidas. Al terminar el proyecto, el grupo deberá poder explicar cómo funciona la estructura que implementó, cómo la integró con PostgreSQL, qué comportamiento observó en las pruebas y qué decisiones tomó al distribuir los datos.

- Implementar una estructura alternativa de indexación y comprender sus operaciones internas.
- Integrar la solución con PostgreSQL de manera funcional y demostrable.
- Comparar experimentalmente la estructura implementada con el acceso sin índice y con B-tree de PostgreSQL.
- Diseñar una estrategia de almacenamiento distribuido considerando fragmentación, asignación, replicación e indexación.
- Analizar los resultados obtenidos y justificar las decisiones tomadas.

2. Organización general

Etapa	Duración	Resultado principal	Evaluación
I. Indexación	5 semanas	Estructura alternativa implementada e integrada funcionalmente con PostgreSQL	Examen parcial práctico
II. Evaluación y distribución	5 semanas	Pruebas de rendimiento + diseño/implementación o simulación distribuida + análisis	Examen final

3. Modalidad de trabajo

El proyecto se realizará en los grupos realizados en clase. El trabajo puede dividirse internamente, pero todos los integrantes deben conocer el funcionamiento general de la solución. En las

evaluaciones se podrán hacer preguntas individuales, revisar partes del código o pedir una prueba sencilla en el momento.

Todo el trabajo deberá mantenerse en un repositorio Git. Si se utiliza código, bibliotecas, repositorios, herramientas de IA u otros recursos externos, estos deberán indicarse claramente. El grupo debe poder distinguir qué reutilizó y qué desarrolló por cuenta propia, y deberá ser capaz de explicar el código que entrega.

4. Estructuras de indexación disponibles

Cada grupo elegirá una de las estructuras indicadas a continuación. Para evitar que todos trabajen el mismo caso, habrá como máximo tres grupos por estructura. B-tree se utilizará como referencia para las comparaciones y no será una opción de implementación.

Estructura	Idea central	Operaciones mínimas al parcial	Consulta principal
Skip List	Estructura ordenada probabilística multinivel	Construcción, búsqueda e inserción	Igualdad; rango según alcance
Radix Tree	Organización de claves a partir de prefijos	Construcción, búsqueda e inserción	Igualdad y prefijo
T-Tree	Árbol balanceado con múltiples valores por nodo	Construcción, búsqueda e inserción	Igualdad; rango según alcance
ISAM	Acceso secuencial indexado con organización por niveles	Construcción, búsqueda e inserción/overflow simplificado	Igualdad y rango
Extendible Hashing	Hashing dinámico mediante directorio y buckets	Construcción, búsqueda, inserción y split básico	Igualdad

Las estructuras no ofrecen exactamente las mismas operaciones, por lo que algunos detalles del trabajo cambian según la opción elegida. En todos los casos se mantendrá un alcance equivalente y manejable.

5. ETAPA I – Indexación (5 semanas)

5.1. Objetivo de esta etapa

En esta primera etapa cada grupo implementará una estructura de indexación distinta de B-tree y la utilizará desde PostgreSQL. El objetivo es comprender cómo se organiza la estructura, cómo se realizan sus operaciones principales y qué implica incorporarla al DBMS.

5.2. Alcance para el examen parcial

- Explicar cómo se organiza la estructura, cuáles son sus operaciones principales, su complejidad, ventajas y limitaciones.
- Implementar código propio para la estructura seleccionada.

- Implementar, como mínimo, las operaciones de construcción, búsqueda e inserción; los requisitos específicos se ajustarán a la estructura elegida.
- Integrar la estructura con PostgreSQL. No será suficiente tener un programa externo que lea un CSV o que solamente envíe consultas a PostgreSQL.
- Demostrar mediante los datos de prueba que la estructura se construye y se utiliza correctamente.
- Mostrar en el código dónde se realizan las operaciones fundamentales y explicar el flujo desde PostgreSQL hacia la estructura implementada.
- Hacer una primera comparación con B-tree. En el parcial se evaluará principalmente que la solución funcione, esté integrada y sea comprendida por el grupo.

5.3. Alcance del trabajo

El alcance está pensado para cinco semanas. Se pide una implementación académica funcional, no un método de acceso completo de PostgreSQL. Por ello no será obligatorio implementar concurrencia completa, WAL propio, recuperación avanzada ante fallos, VACUUM especializado, estimación sofisticada de costos, soporte completo de unicidad ni todas las operaciones posibles de un método de acceso. Si un grupo desea trabajar alguno de estos aspectos, podrá hacerlo como ampliación una vez que tenga completo el alcance obligatorio.

5.4. Cronograma de la primera etapa

Semana	Objetivo	Avance esperado
1	Comprensión y diseño	Propuesta aprobada, referencias, representación de la estructura, pseudocódigo y estrategia de integración
2	Implementación inicial	Construcción y búsqueda funcionando con pruebas
3	Operaciones principales	Construcción + búsqueda + inserción; demo breve de código
4	Integración PostgreSQL	Primera demostración funcional de uso desde PostgreSQL y corrección de errores
5	Parcial práctico	Demo reproducible, código, defensa individual y comparación preliminar con B-tree

5.5. Propuesta inicial – Semana 1 (2-09-26)

Documento de máximo 2 páginas que incluya:

- Estructura elegida y motivación;
- Referencias fundamentales y al menos una fuente científica o técnica de calidad;
- Representación de la estructura y operaciones a implementar;
- Tipo de clave y consultas soportadas;
- Estrategia de integración con PostgreSQL;
- Identificación de qué componentes serán desarrollados por el equipo;
- Recursos externos, repositorios o bibliotecas que planean consultar o reutilizar;

- Riesgos técnicos identificados y plan de trabajo del grupo.

5.6. Evaluación del parcial

La evaluación se realizará utilizando el entorno reproducible del propio grupo. Durante la presentación se podrá pedir, por ejemplo, buscar una clave existente o inexistente, insertar nuevos datos, reiniciar el entorno, explicar una función del código o ejecutar una prueba adicional. La finalidad es comprobar que la solución funciona y que el grupo comprende lo que ha implementado.

6. ETAPA II – Evaluación experimental y distribución (5 semanas)

6.1. Objetivo de esta etapa

En la segunda etapa se evaluará con mayor detalle el comportamiento del índice y luego se trabajará con un escenario distribuido. El grupo deberá relacionar las decisiones de indexación, fragmentación, asignación y replicación con el comportamiento observado en las pruebas.

6.2. Evaluación del índice

Todos los grupos generarán sus datos y cargas de trabajo siguiendo las mismas especificaciones. De esta manera las comparaciones tendrán condiciones comunes. Se podrán realizar pruebas adicionales, pero no en reemplazo de las indicadas en estas pautas.

Escenario	Obligatorio
Acceso sin índice	Sí
B-tree de PostgreSQL	Sí
Estructura implementada	Sí
Al menos tres tamaños de dataset	Sí
Múltiples ejecuciones por experimento	Sí
Carga de trabajo favorable y otra no favorable o neutra	Sí

Las métricas mínimas serán: tiempo de construcción, tiempo de búsqueda, comportamiento de inserción, tamaño del índice y evolución al aumentar el volumen de datos. Cuando corresponda a la estructura, se incorporarán búsquedas por rango o prefijo.

6.3. Trabajo con datos distribuidos

Para la parte distribuida se trabajará con tres nodos y con los parámetros indicados en estas pautas. Cada grupo levantará y documentará su propio entorno. No se pide construir un DBMS distribuido desde cero; se pide aplicar los conceptos mínimos y justificar las decisiones tomadas.

- Definir y justificar cómo se fragmentarán los datos.
- Indicar en qué nodo se almacenará cada fragmento.
- Decidir qué datos se replicarán y explicar por qué.
- Probar consultas locales y consultas que necesiten acceder a más de un nodo.
- Comparar al menos dos estrategias de distribución o asignación.
- Observar cómo afectan la transferencia de datos y la latencia a las consultas.
- Evaluar dónde conviene utilizar B-tree y/o la estructura desarrollada en la primera etapa.
- Probar al menos un cambio en la carga de trabajo y la indisponibilidad de un nodo.

6.3.1. Escenario común de distribución

Se utilizarán tres nodos PostgreSQL en contenedores Docker: A, B y C. Para las pruebas se considerarán las siguientes latencias entre nodos: A-B = 20 ms, A-C = 40 ms y B-C = 25 ms. El grupo deberá explicar cómo incorporó estos valores en su experimento o simulación.

Cada grupo construirá un esquema sencillo (puede estar basado en su trabajo de BD1) con al menos cuatro relaciones vinculadas (por ejemplo: clientes, productos, pedidos y detalle de pedido). La relación principal utilizada en las pruebas de distribución deberá tener al menos 1 000 000 de registros. A partir de este esquema, el grupo propondrá la fragmentación, la asignación y la replicación de los datos y comparará al menos dos alternativas.

Las pruebas deberán considerar una carga aproximada de 70% de lecturas y 30% de escrituras, consultas locales y consultas que involucren más de un nodo. También se probará qué ocurre cuando un nodo no está disponible y cuando cambia la concentración de accesos entre nodos. Los supuestos y la forma de generar las operaciones deberán quedar documentados.

6.4. Cronograma de la segunda etapa

Semana	Objetivo	Avance esperado
6	Pruebas de rendimiento	Experimentos controlados: sin índice vs. B-tree vs. estructura propia
7	Diseño distribuido	Fragmentación, asignación, replicación y justificación
8	Implementación/simulación	Nodos y consultas distribuidas funcionando en el entorno definido
9	Integración y experimentos	Comparación de estrategias y análisis de resultados
10	Evaluación final	Sistema, resultados, conclusiones, repositorio y defensa

7. Datasets y workloads

Familia de datos	Propósito
Claves numéricas secuenciales	Observar crecimiento e inserciones ordenadas
Claves numéricas aleatorias	Comparación común de búsquedas e inserciones
Distribución sesgada	Analizar sensibilidad a distribución y frecuencia de acceso
Cadenas	Evaluar estructuras orientadas a claves textuales
Cadenas con prefijos compartidos	Analizar Radix Tree y estructuras relacionadas

Cada grupo generará los datos que utilizará en las pruebas. No se entregarán archivos de datos preparados. Los generadores deberán quedar en el repositorio, de modo que cualquier integrante pueda volver a producir los mismos datos utilizando los parámetros indicados a continuación.

Universidad Católica San Pablo

7.1. Datasets communes

ID	Datos	Tamaños obligatorios	Especificación
D1	INTEGER secuencial	100 000; 500 000; 1 000 000	Claves únicas 1..N.
D2	INTEGER aleatorio	100 000; 500 000; 1 000 000	Claves enteras aleatorias, únicas y generadas de forma reproducible. El grupo deberá documentar el procedimiento y la semilla utilizada.
D3	INTEGER con acceso sesgado	100 000; 500 000; 1 000 000	Datos D2; carga de trabajo de consultas con distribución Zipf o equivalente. El grupo deberá documentar el procedimiento y los parámetros utilizados para su generación.
D4	VARCHAR aleatorio	100 000; 500 000; 1 000 000	Cadenas alfanuméricas generadas de forma reproducible, con longitud fija o acotada. El grupo deberá documentar el procedimiento y los parámetros utilizados para su generación.
D5	VARCHAR con prefijos	100 000; 500 000; 1 000 000	Cadenas agrupadas por prefijos compartidos, generadas de forma reproducible.

No todos los conjuntos de datos son igualmente útiles para todas las estructuras. Cada grupo utilizará los que correspondan a su caso, pero deberá incluir al menos un conjunto que permita comparar su implementación con B-tree. Radix Tree deberá trabajar con el conjunto de cadenas con prefijos compartidos; las estructuras orientadas a claves numéricas deberán incluir el conjunto de enteros aleatorios.

7.2. Carga de trabajo minima

ID	Operación	Cantidad mínima
W1	Búsquedas exitosas por igualdad	10 000 claves seleccionadas reproduciblemente
W2	Búsquedas no exitosas	10 000 claves ausentes
W3	Inserciones	10% adicional respecto del tamaño inicial del dataset
W4	Búsquedas por rango	1 000 consultas, cuando la estructura soporte naturalmente esta operación
W5	Búsquedas por prefijo	1 000 consultas para Radix Tree y estructuras donde corresponda

Las operaciones de prueba también deberán generarse de manera reproducible. Para una misma comparación se usarán exactamente los mismos datos y las mismas operaciones en los tres casos: sin índice, con B-tree y con la estructura implementada por el grupo.

8. Entorno de trabajo y reproducibilidad

Cada grupo será responsable de preparar y mantener su entorno de trabajo. La docente no entregará contenedores preconfigurados ni realizará configuraciones particulares para los grupos. Todo lo necesario para ejecutar el proyecto deberá quedar en el repositorio.

- DBMS: PostgreSQL 18.6.
- Ejecución: Docker sobre contenedores Linux.
- Lenguaje: C para los componentes que requieran integración interna con PostgreSQL; podrán utilizar scripts auxiliares en otro lenguaje para generación de datos, automatización y análisis.
- Control de versiones: Git.
- Repositorio: deberá contener todo lo necesario para reconstruir el proyecto desde cero.
- Archivos mínimos del entorno: Dockerfile y, cuando corresponda, docker-compose.yml; scripts de compilación/instalación; scripts SQL; generadores de datos; workloads; scripts de experimentación y README.
- La solución deberá poder construirse y ejecutarse siguiendo únicamente las instrucciones del README. No se evaluará sobre un entorno configurado manualmente que no pueda reproducirse desde el repositorio.
- Todos los experimentos oficiales deberán registrar versión de PostgreSQL, configuración relevante, tamaño del dataset, número de repeticiones y condiciones de ejecución.
- Cada medición deberá repetirse al menos 5 veces. El grupo reportará una medida central (media o mediana) y justificará su elección; no se aceptarán conclusiones basadas en una sola ejecución.

9. Informe final

El informe deberá contener, como mínimo:

- Introducción y motivación.

- Fundamentos de la estructura seleccionada y comparación conceptual con B-tree.
- Diseño e implementación.
- Integración con PostgreSQL.
- Metodología experimental.
- Resultados del benchmark.
- Diseño distribuido: fragmentación, asignación y replicación.
- Resultados del escenario distribuido.
- Discusión: cuándo funciona mejor o peor cada alternativa y por qué.
- Limitaciones y amenazas a la validez.
- Conclusiones que respondan las preguntas orientadoras.
- Referencias y enlace al repositorio reproducible.

10. Criterios generales de evaluación

Dimensión	Qué se observará
Comprensión	Capacidad para explicar estructura, decisiones y comportamiento sin depender de diapositivas
Implementación	Código propio, corrección de operaciones y calidad técnica
Integración	Evidencia de que PostgreSQL utiliza/interactúa realmente con la solución
Experimentación	Diseño controlado, repetibilidad, métricas y comparaciones justas
Análisis	Interpretación de resultados; no basta reportar tiempos
Diseño distribuido	Justificación de fragmentación, asignación, replicación y transferencia
Reproducibilidad	Repositorio, scripts, documentación y posibilidad de repetir las pruebas
Defensa individual	Comprensión del proyecto completo y capacidad de responder/modificar en vivo

11.Consideraciones importantes

- No se espera que la estructura implementada sea siempre más rápida que B-tree. Encontrar casos en los que tenga peor comportamiento también es un resultado válido, siempre que las pruebas estén bien realizadas y el grupo pueda explicar por qué ocurre.
- No se aceptarán comparaciones con datasets, consultas o condiciones diferentes sin una justificación experimental.
- Todo código reutilizado debe ser declarado y citado. El equipo debe identificar con claridad su contribución propia.
- La docente podrá solicitar la explicación de cualquier fragmento del código y realizar pruebas adicionales durante la defensa.
- El trabajo grupal no reemplaza la evaluación individual: todos los integrantes deben comprender el sistema completo.
- Se valorará más una solución correcta, entendida y reproducible que una implementación innecesariamente compleja que el grupo no logre terminar o explicar.

12.Resultado esperado

Al finalizar el proyecto, cada estudiante deberá poder explicar el funcionamiento de la estructura trabajada, mostrar cómo se utilizó con PostgreSQL, interpretar los resultados obtenidos y justificar las decisiones de fragmentación, asignación, replicación e indexación tomadas en el escenario distribuido.